Part D: State-Feedback Controller

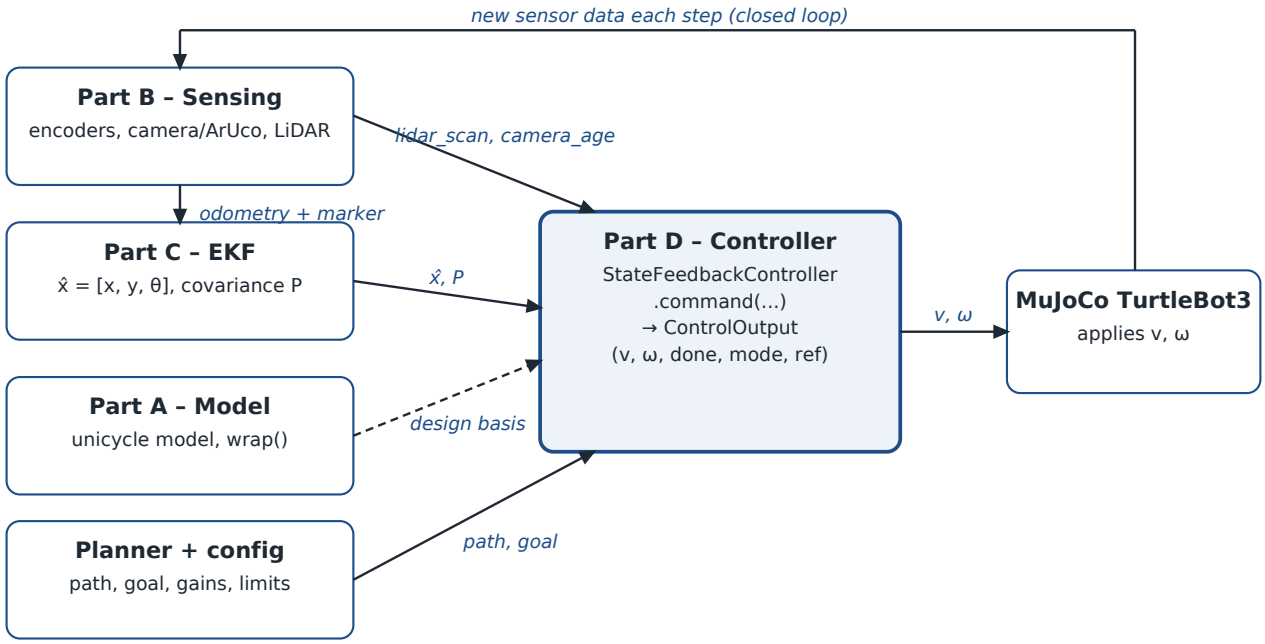
41014 Integrated Project, Track A (TurtleBot3 in MuJoCo). A walkthrough of `controller.py`: what each block does, the maths behind it, how it plugs into the rest of the pipeline, and what to say about it in the viva.

1. What this file does, in one paragraph

Every control cycle, `StateFeedbackController.command()` receives the EKF's estimated pose $\hat{x}$ = [x, y, θ] and its covariance P, the planned path, the latest LiDAR scan and how long ago the ArUco marker was last seen. It first checks that it can trust its inputs (finite estimate, small enough uncertainty, valid LiDAR). It then picks one of four behaviours depending on where the robot is: **turn to path**, **track** the path, **approach** the dock, or **align** at the dock. Finally, a LiDAR safety layer slows or stops forward motion near obstacles. It returns a linear velocity v, angular velocity ω, a *done* flag and a *mode* label for logging.

> **Key viva point:** the controller only ever sees $\hat{x}$ and P from the EKF, never simulator ground truth. That is the brief's hard constraint for Track A. Ground truth should appear only in the evaluation/logging code.

2. Where it sits in the pipeline



Section	What it gives the controller	Why the controller needs it
Part A – Modelling	Unicycle kinematics ($\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$) and the `wrap()` helper	The control law is designed for this model: it can only command v and ω, and cannot move sideways (non-holonomic).
Part B – Sensing	LiDAR ranges, camera/ArUco detections, encoder ticks	LiDAR feeds the safety layer directly. Camera freshness (`camera_age`) gates docking completion.
Part C – Estimation	$\hat{x}$ and P from the EKF	$\hat{x}$ is the state being fed back. P is used to refuse to move when the estimate is untrustworthy.
Planner / config	Path waypoints, goal pose, gains, limits, tolerances	Defines the reference to track and all tuning values.

Section	What it gives the controller	Why the controller needs it
Part E – Integration	Uses `ControlOutput.mode` and `reference` in logs	Mode labels make failures diagnosable (e.g. lots of *uncertainty_stop* points to the EKF, not the controller).

3. Inputs and outputs

Inputs to `command(x_hat, P, path, lidar_scan, camera_age)`

Argument	Shape / type	Meaning
x_hat	(3,) float	EKF estimate [x, y, θ] in the world/map frame (m, m, rad)
P	(3, 3) float	EKF covariance. Diagonal entries are variances of x, y, θ
path	(N, 2) float	Planned waypoints from the start to near the dock, in the same frame as $\hat{x}$
lidar_scan	(lidar_beams,) float	Range per beam in m. inf/nan for no return
camera_age	float (s)	Time since the last valid ArUco detection

Output: `ControlOutput` (lines 15–29)

Field	Meaning
v, omega	Commanded linear (m/s) and angular (rad/s) velocity, sent to /cmd_vel or the MuJoCo actuator
done	True only once docked and held for 10 consecutive good cycles
mode	Which behaviour produced the command, plus +slow / +stop if the safety layer acted
reference	The pose the controller was steering toward this cycle. Log it to plot reference vs estimate

Config values read from `self.c`

Parameter	Used for
goal (x, y, θ)	Docking pose in front of the ArUco charging station
lookahead	Distance ahead on the path to track. Also the radius at which docking mode starts
k1, k2, k3	Tracking gains on forward error, lateral error and heading error
max_v, max_omega	Actuator limits (TurtleBot3 Burger: 0.22 m/s, 2.84 rad/s)
goal_position_tolerance, goal_heading_tolerance	When the robot counts as "at" the dock
camera_max_age	How recent a marker sighting must be to accept docking
lidar_beams, lidar_angle_min, lidar_angle_inc	LiDAR geometry, to compute each beam's angle

4. Line-by-line walkthrough

Lines 10–12: imports

`wrap` comes from `model.py` (the Part A model module). It maps any angle into $[-\pi, \pi]$ so heading errors take the short way round. Without it, an error of 350° would make the robot spin almost a full turn instead of 10° the other way.

$$\text{wrap}(a) = \text{atan2}(\sin a, \cos a)$$

Lines 41–53: `__init__` and `reset`

The controller keeps only two pieces of memory between cycles. `turning` is the hysteresis flag for turn-to-path mode, and `dwell` counts consecutive good docking cycles (10 required). `reset()` clears both before each trial, so repeated trials are independent, which matters for the repeated-trial evaluation in the Distinction band.

Lines 55–58: `_closest_index`

Finds the path point nearest the robot by minimising squared distance (no square root needed, since it doesn't change the argmin):

$$i^* = \operatorname{argmin}_j \|p_j - \hat{p}\|^2$$

Lines 60–88: `_lookahead_reference`

Builds the reference the robot steers toward: a point a fixed distance L (`lookahead`) along the path ahead of the robot. Starting from the closest point, it walks segment by segment, subtracting each segment's length from the remaining distance. When a segment is longer than what remains, it interpolates linearly to land exactly on the lookahead point:

$$p_{target} = p_{prev} + (L_{remain} / \|p_j - p_{prev}\|) \cdot (p_j - p_{prev})$$

If the path runs out first, the target is the last path point. The reference heading is the bearing from the robot to the target:

$$\psi_{ref} = \operatorname{atan2}(y_t - \hat{y}, x_t - \hat{x})$$

Two edge cases are handled: an empty path falls back to the goal (line 67), and a target sitting on the robot uses the goal heading to avoid atan2(0, 0) (line 85). Tracking a point ahead, instead of the nearest point, smooths the motion and cuts corners slightly, like pure pursuit.

Lines 90–108: `_front_clearance`

Reduces the LiDAR scan to one number: the closest obstacle in a ±28° cone in front of the robot. Each beam's angle is:

$$\varphi_i = \varphi_{min} + i \cdot \Delta\varphi, \text{ front if } |\text{wrap}(\varphi_i)| \le 28°, \text{ clearance} = \min_{front} r_i$$

It returns **nan** (treated as a failure) if the scan has the wrong number of beams or no valid returns at all, and **inf** if the scan is valid but nothing is in the forward cone. 28° is a bit wider than the robot's half-width at the stopping distance, so the cone covers the body's swept path rather than just the centreline.

Lines 117–134: guard layer (trust your inputs first)

Before computing any motion, three checks can force a stop ($v = \omega = 0$):

- **invalid_estimate**: $\hat{x}$ or P contains nan/inf, meaning the EKF has diverged.
- **uncertainty_stop**: the 1-σ position uncertainty is above 0.45 m:

$$\sigma_{pos} = \sqrt{(P_{xx} + P_{yy})} > 0.45 \text{ m}$$

- **lidar_failure**: the clearance is nan, so the robot cannot guarantee it won't hit something.

This is the main link between estimation and control: the controller uses P, not just $\hat{x}$. That is a good answer to "what happens when the marker leaves view?": odometry-only prediction makes P grow, and once it passes the threshold the robot stops rather than drive on a bad estimate.

Lines 136–138: goal geometry

$$d_{goal} = \|g_{xy} - \hat{p}\|, \; e_{\theta,goal} = \text{wrap}(\theta_g - \hat{\theta})$$

Lines 141–158: dock_align (inside position tolerance)

The robot is close enough, so it stops translating and only rotates to the docking heading. ω is limited to ±0.7 rad/s for a gentle final turn:

$$v = 0,\ \omega = \mathrm{clip}(2.0 \cdot e_{\theta,\mathrm{goal}},\ -0.7,\ 0.7)$$

A cycle counts as "good" only if the heading is within tolerance **and** the camera saw the marker recently. After 10 consecutive good cycles, done is set. Requiring a fresh camera sighting makes docking genuinely visual: the robot can't declare success from drifting odometry alone. The dwell stops a single lucky reading from ending the trial.

Lines 159–165: dock_approach (within lookahead of the goal)

The robot stops following the path and drives straight at the goal pose, slowly:

$$\beta = \mathrm{wrap}(\mathrm{atan2}(\Delta y,\ \Delta x) - \theta)$$

$$v = \min(0.10,\ 0.6 \cdot d_{\mathrm{goal}}) \cdot \max(0,\ \cos\beta)$$

$$\omega = \mathrm{clip}(2.2 \cdot \beta + 0.35 \cdot e_{\theta,\mathrm{goal}},\ \pm\omega_{\max})$$

Speed is capped at 0.10 m/s and shrinks linearly near the goal. The *max(0, cos β)* factor means the robot only drives forward when it is facing the goal, and stops translating if the goal is behind it. ω mostly points at the goal (2.2β) with a small pull toward the final docking heading ($0.35 \cdot e_\theta$), so it arrives roughly aligned and dock_align has less to correct.

Lines 166–193: track and turn_to_path

Far from the goal, the robot follows the path. First the error to the lookahead reference is rotated into the robot's body frame:

$$[e_x,\ e_y]^T = R(-\theta) \cdot [\Delta x,\ \Delta y]^T\text{: } e_x = \cos\theta\,\Delta x + \sin\theta\,\Delta y,\ e_y = -\sin\theta\,\Delta x + \cos\theta\,\Delta y$$

e_x is how far ahead the target is and e_y is how far to the left. The body frame matters because the robot can only act along its own forward axis (v) and rotate (ω). It also computes the heading error e_θ and the bearing to the target α.

Hysteresis (lines 176–180). If $|\alpha| > 0.65$ rad (≈37°) the robot enters **turn_to_path** and rotates on the spot ($v = 0$, $\omega = 1.8\alpha$). It only leaves when $|\alpha| < 0.35$ rad (≈20°). Two thresholds instead of one stop the controller chattering between modes when α sits near a single cut-off.

Tracking law (lines 186–193). A modified Kanayama controller:

$$v_{\mathrm{ref}} = 0.7 \cdot v_{\max}$$

$$v = (v_{\mathrm{ref}} \cos e_\theta + k_1\, e_x) \cdot \cos^2\alpha,\ v \in [0,\ v_{\max}]$$

$$\omega = k_2\, v_{\mathrm{ref}}\, e_y + k_3 \sin e_\theta,\ \omega \in [-\omega_{\max},\ \omega_{\max}]$$

$v_{\mathrm{ref}} \cos e_\theta$ is the feedforward cruise speed. $k_1 e_x$ corrects forward error, $k_2 v_{\mathrm{ref}} e_y$ steers out lateral error, and $k_3 \sin e_\theta$ corrects heading. The $\cos^2\alpha$ factor slows the robot smoothly as it gets misaligned, so it turns more and travels less when off course. v is clipped at 0, so the robot never reverses. Turn-to-path handles large misalignment instead.

Because $v_{\mathrm{ref}} > 0$ is always used here, the k_2 lateral term is active. That is what lets the robot correct sideways error, unlike a stationary-waypoint version where $v_{\mathrm{ref}} = 0$ removes it.

Lines 195–205: LiDAR safety layer

Applied after whichever mode ran, so it overrides every behaviour. It only changes v; ω is left alone so the robot can still rotate away from an obstacle.

$$c < 0.24\text{ m: } v = 0\ (+\mathrm{stop})\quad 0.24 \le c < 0.50\text{ m: } v \leftarrow v \cdot (c - 0.24) / 0.26\ (+\mathrm{slow})$$

The scale goes linearly from 0 at 0.24 m to 1 at 0.50 m, so braking is smooth rather than a sudden stop. 0.24 m is roughly the Burger's radius plus a margin. Note this layer does not plan a way around obstacles; it relies on the planner's path avoiding them, and only protects against collisions from tracking error or estimate drift.

5. Mode summary

Mode	When	v	ω
invalid_estimate	$\hat{x}$ or P not finite	0	0
uncertainty_stop	$\sqrt{(P_{xx}+P_{yy})} > 0.45$ m	0	0
lidar_failure	Bad scan length or no valid returns	0	0
turn_to_path	Far from goal, $\lvert\alpha\rvert > 0.65$ (until < 0.35)	0	1.8α
track	Far from goal, aligned	$(v_{ref}\cos e_\theta + k_1 e_x)\cos^2\alpha$	$k_2 v_{ref} e_y + k_3 \sin e_\theta$
dock_approach	$d_{goal} \le$ lookahead	$\min(0.1, 0.6d)\cdot\max(0, \cos\beta)$	$2.2\beta + 0.35 e_{\theta,goal}$
dock_align	$d_{goal} \le$ position tolerance	0	$\text{clip}(2e_{\theta,goal}, \pm 0.7)$
docked	10 good cycles (heading OK + fresh camera)	0	0
…+slow / +stop	Clearance < 0.50 / < 0.24 m	scaled / 0	unchanged

6. Linking to the model (Part A) and justifying gains

The brief asks for a state-feedback or LQR tracker. The tracking law can be justified by linearising the body-frame error dynamics of the unicycle about the reference motion (v_r, ω_r):

$$\dot{e} = A\,e + B\,u,\quad e = [e_x, e_y, e_\theta]^T$$

$$A = [[0, \omega_r, 0], [-\omega_r, 0, v_r], [0, 0, 0]],\quad B = [[1, 0], [0, 0], [0, 1]]$$

This system is controllable whenever $v_r \neq 0$, which is exactly why the tracker keeps a nonzero v_{ref}. With $v_r = 0$, e_y cannot be influenced by any linear feedback (Brockett's condition), so this file handles the final docking separately with approach-then-align. Choosing Q and R and solving the LQR problem gives a gain matrix K whose entries map onto k_1, $k_2 v_{ref}$ and k_3. That turns the gains from "tuned by hand" into "derived from the model and defended", which is what the Credit and Distinction bands ask for.

Useful trade-offs to quantify for Distinction: lookahead L (larger is smoother but cuts corners more), k_2 (higher removes lateral error faster but oscillates and saturates ω more often), and the 0.45 m uncertainty threshold (lower is safer but stops more often when the marker is out of view).

7. Honest limitations (good for the "limits of the approach" discussion)

- **e_θ and α are the same thing in track mode.** The reference heading is defined as the bearing from the robot to the lookahead point (line 84), so $e_\theta = \alpha$ and $e_y = d\cdot\sin\alpha$. The "Kanayama" law therefore behaves like pure pursuit, steering at the lookahead point, rather than matching the path's tangent direction. That works, but it's worth knowing if asked what e_θ means here.
- **Possible deadlock in uncertainty_stop.** If the robot stops with the marker out of view, the EKF may keep adding process noise without a correction, so P never shrinks and the robot stays stopped. A recovery behaviour (e.g. rotate in place to search for the marker) would fix

this.

- **Hard-coded numbers.** 0.45, 0.24, 0.50, 28°, 2.2, 0.35, 1.8, 2.0, 0.7, 0.10, 0.6 and the dwell of 10 live in the code, not the config. Moving them into config makes sweeps easy and each one needs a stated reason.
- **Dwell depends on control rate.** 10 cycles is 0.5 s at 20 Hz but 1 s at 10 Hz. Defining it in seconds is more robust.
- **Safety layer only brakes.** It cannot steer around obstacles and ignores beams outside the ±28° cone, so it assumes rotating on the spot is always safe.
- **No reversing.** v is clipped at 0, so overshooting the dock can only be fixed by turning around.

8. What to log and report (Part D results)

Metric	How	Why
RMS cross-track and heading error	Ground-truth pose vs reference, track mode only	Main tracking-performance number
Final docking error	Ground-truth position and heading error when done = True	Shows whether the task actually succeeded
Docking success rate and time	Over repeated trials with varied start poses	Distinction-level systematic evaluation
Time in each mode	Count mode labels	Diagnoses failures, e.g. many uncertainty_stop → EKF problem
Saturation fraction	% cycles where |v| or |ω| hits the limit	Shows whether gains are too aggressive
Control effort	$\sum(v^2 + \omega^2)\cdot\Delta t$	Effort vs tracking error trade-off

Use ground truth **only** in this evaluation code, and make that visible in the repo: the marking guide checks for it.

9. Likely viva questions, with short answers

Q: Why rotate the error into the robot frame?

Because a differential-drive robot can only act along its forward axis and rotate. In the body frame, e_x maps directly to v and e_y/e_θ to ω.

Q: What happens when the marker leaves the camera view?

The EKF runs on odometry only, so P grows. If $\sqrt{(P_{xx}+P_{yy})}$ passes 0.45 m the controller stops. At the dock, docking can't complete without a fresh marker sighting.

Q: Why use a lookahead point instead of the nearest point?

It gives a reference the robot is always moving toward, keeps $v_{ref} > 0$ meaningful, and smooths the path. Larger L is smoother but cuts corners more.

Q: Why two thresholds for turn_to_path?

Hysteresis. With one threshold, noise in α near the cut-off would make the robot flick between modes every cycle.

Q: How were the gains chosen and what limits them?

Ideally LQR on the linearised error model with Q from acceptable errors and R from velocity limits. They are limited by actuator saturation and by EKF noise: high gains amplify estimate noise into jittery commands.

Q: If the EKF were perfect, would docking be perfect?

No. Tolerances, the dwell window, saturation and the no-reverse constraint still limit final accuracy. But in practice, estimate error is usually the biggest contributor, since the controller tracks $\hat{x}$ not the true pose.

Q: Why does the safety layer only change v?

So the robot can still rotate to escape. If ω were also zeroed, the robot could get stuck facing an obstacle.